

Backing Store

1 Core Idea

Definition

Backing store is the disk space used to hold pages when they are not currently in main memory.

Page replacement decides *which* page leaves memory. Backing-store management decides *where* that page is kept on disk and how the operating system finds it again later.

Main Question

When a page is evicted from RAM, where exactly is its disk copy stored?

2 Swap Area

Many systems reserve a special disk region called a **swap partition** or **swap area**.

Feature	Meaning
No normal file system	The OS can use raw block numbers directly.
Lower overhead	No need to translate file offsets into disk blocks.
Separate disk possible	Paging I/O can be separated from file-system I/O.
Free-list management	Free swap chunks are tracked and reused as processes exit.

At boot time, the swap partition is empty. The OS can represent it as one large free region with a starting block and a size.

3 Static Swap Allocation

In the simplest design, each process gets a fixed swap area large enough for its whole virtual image.

Static Swap Address

$$\text{disk address of page } i = \text{start of process swap area} + i$$

Here i is the virtual page number. If the process swap area starts at disk page 1000, then virtual page 5 is stored at disk page 1005.

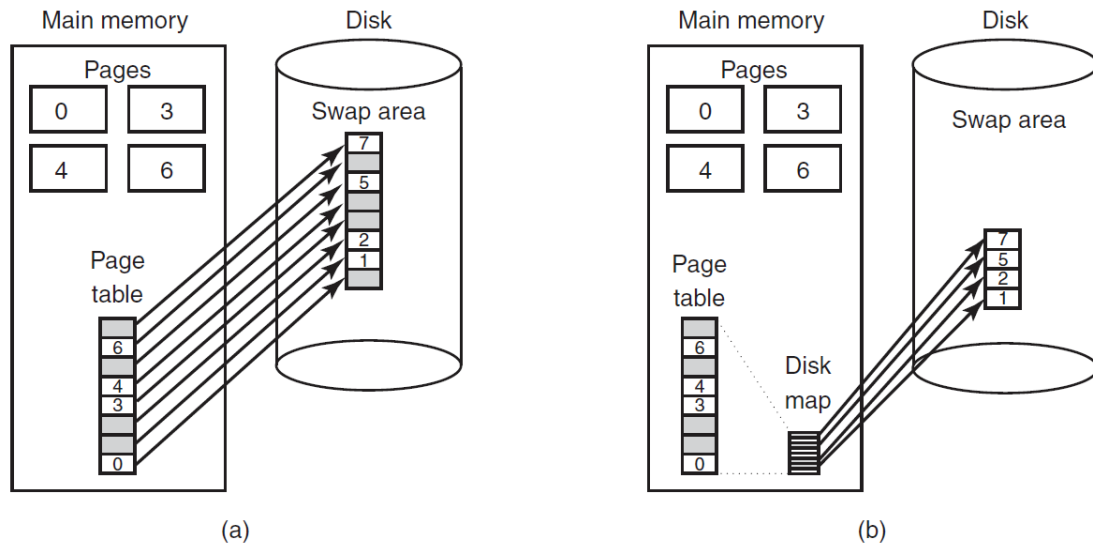


Figure 3-28. (a) Paging to a static swap area. (b) Backing up pages dynamically.

Figure 3-28(a)

In static backing, every virtual page has a fixed disk slot. Pages 0, 3, 4, and 6 are in memory, but the disk area still reserves slots for all pages 0 through 7.

4 How Static Backing Works

State	Disk meaning
Page absent from RAM	Its fixed disk slot is used to bring it back later.
Page present in RAM and clean	The old disk copy is still valid.
Page present in RAM and dirty	The disk copy may be stale and must be overwritten if the page is evicted.

This design makes address calculation simple because the disk location depends only on the process' swap-area start and the virtual page number.

Limitation

Static allocation can waste swap space because a process reserves disk space even for pages that are currently in memory.

5 Growth Problem

Processes may grow after they start:

- program text is usually fixed;

- the data segment can grow, for example because of heap allocation;
- the stack can grow as function calls and local variables accumulate.

If one fixed swap area was reserved at process creation time, it may become too small later. A better design may reserve separate backing areas for text, data, and stack, or allow each area to be made from multiple disk chunks.

6 Dynamic Backing Allocation

The opposite approach is to allocate no swap space in advance. A disk slot is chosen only when a page is actually swapped out.

Figure 3-28(b)

In dynamic backing, pages 1, 2, 5, and 7 are on disk, but they do not need fixed positions based on their virtual page numbers. A disk map records where each swapped-out page is stored.

Method	Advantage	Cost
Static swap area	Very simple address calculation.	Reserves disk space even when pages are in RAM.
Dynamic backing	Uses swap space only for pages that are actually on disk.	Needs a per-process disk map to remember each page's disk location.

7 Swap Partition vs Swap File

Backing type	Explanation
Swap partition	Dedicated disk partition used only for paging. Common in UNIX-style systems.
Swap file	Large preallocated file inside the normal file system. Common in Windows-style systems.

A swap partition avoids normal file-system overhead. A swap file is easier to create when repartitioning the disk is not possible.

8 Executable Files as Backing Store

Program text pages usually come from an executable file and are normally read-only. This creates an optimization:

- clean program-text pages can be discarded instead of written to swap;
- if needed again, they can be reread from the executable file;
- shared-library text pages can use the same idea.

Why This Saves Space

Read-only code already has a valid copy in the executable file, so the OS does not need to store another copy in the swap area.

9 Final Summary

One-Box Summary

Backing store holds pages that are not in RAM.
Static backing gives each virtual page a fixed disk slot.
Dynamic backing allocates disk slots only when pages are evicted.
Clean code pages can often be reread from executable files.